



Per-LLM Chains

One Sovereign Chain per Model:

Unified Training, Research, and

Pooled GPU Hosting on Quasar 3.0

Antje Worring & Zach Kelling

Zoo Labs Foundation

December 15, 2025

Abstract

We introduce the *Per-LLM Chain* architecture: a sovereign QUASAR-certified appchain dedicated to the lifetime of a single large language model. Each chain binds together training-data provenance, compute receipts, research artifact registries, contributor attribution graphs, and inference revenue distribution into a single consensus-ordered ledger. Per-LLM chains settle on LUX QUASAR 3.0 (LP-020) via the QUASAR-Native App Stack (LP-133), use the QUASARGPU adapter (LP-132) for deterministic GPU scheduling, anchor TEE-attested compute receipts to the LUX A-Chain (LP-134), and federate cloud capacity through LUX Cloud (LP-136). The canonical deployment hosts the ZEN model family—eighteen open-weight frontier models from **zen4-nano** (0.8B) through **zen4-ultra-max** (1T+)—each with its own chain. The ZOO per-LLM stack complements the HANZO AI Chain (which operates the inference and commerce marketplace) by providing the upstream training, research, and capacity layer. Spec freeze: 2025-12-15. Activation alongside QUASAR 3.0: 2025-12-25.

Keywords: per-LLM chains, decentralized training, GPU pooling, model provenance, QUASAR 3.0, appchain, attribution graphs, ZEN models

Contents

1	Introduction	5
1.1	Why one chain per LLM, not one chain for all LLMs	5
1.2	Position vs centralized AI labs	5
1.3	Contributions	6
2	Architecture	6
2.1	Settlement on QUASAR 3.0	6
2.2	State schema	6
2.2.1	Weights tree	7
2.2.2	Training-data tree	7
2.2.3	Compute-receipts tree	7
2.2.4	Attribution tree	7
2.3	Validator set	7
2.4	Native instructions	7
2.5	System diagram	8
3	Unified Training	8
3.1	Training as transactions	8
3.2	Training-step transaction	8
3.3	Determinism and reproduction	9
3.4	Capacity scheduling via QUASARGPU	9
3.5	Aggregation	9
3.6	Attribution from gradient receipts	9
3.7	Slashing	10
4	Unified Research	10
4.1	Research as on-chain artifacts	10
4.2	Research artifact registry contract	10
4.3	Reproducibility from artifacts	11
4.4	Citation graph	11
4.5	Research credit attribution	11

5	Pooled GPU Hosting	11
5.1	Capacity is staked, not just announced	11
5.2	Federation through LUX Cloud	12
5.3	SLA and slashing	12
5.4	Pricing	12
5.5	Hosting beyond training	12
5.6	Operator economics	12
6	The Zen Model Family	13
6.1	Canonical deployment	13
6.2	Small line (Qwen3.5 base, Gated DeltaNet, 262K context)	13
6.3	Medium line (Qwen3.5 base, mixed dense/MoE)	13
6.4	Coder line	13
6.5	Frontier line	13
6.6	Vision-language sub-family	14
6.7	Speech sub-family	14
6.8	Reranker, embedding, and specialty chains	14
6.9	Lineage	14
6.10	License and attribution	15
7	Cross-Chain Interaction	15
7.1	LUX C-Chain (general EVM)	15
7.2	HANZO AI Chain (inference marketplace)	15
7.3	LUX F-Chain (encrypted inference)	15
7.4	LUX A-Chain and B-Chain (LP-134)	16
7.5	LUX Cloud (LP-136)	16
7.6	Topology summary	16
8	Token Economics	16
8.1	Two-token model	16
8.2	Per-chain token supply	17
8.3	Emission schedule	17
8.4	Inference revenue split	17
8.5	Staking and governance	17
8.6	Slashing economics	17
9	Privacy and Provenance	18
9.1	Selective disclosure of training data	18
9.2	Differential privacy on gradient submission	18
9.3	TEE attestation as compute provenance	18
9.4	Redaction and right-to-revoke	18
9.5	Reading the provenance graph	19
10	Activation	19
10.1	Spec freeze and activation timeline	19
10.2	Genesis state	19
10.3	Operator onboarding	19
10.4	Post-activation roadmap	20

10.5 Conclusion 20

1 Introduction

Foundation-model development is a coordination problem disguised as a compute problem. The visible cost of training a frontier model—tens of millions of dollars in GPU-hours—obscures four invisible coordination failures that no centralized lab has solved:

1. **Training-data provenance.** Who contributed which examples? Were they licensed, consented, or scraped? Can a contributor revoke participation? In a centralized lab the answer is a spreadsheet, a contract, and a hope that nobody asks for receipts.
2. **Compute provenance.** Which GPUs ran which gradient steps? Was the reported loss the loss that actually occurred? In a centralized lab the answer is internal logs that cannot be audited externally.
3. **Research credit attribution.** Whose ablation produced the architecture variant that shipped? Which intern’s hyperparameter sweep found the working learning rate? In a centralized lab the answer is whatever the publication’s author list says.
4. **Inference revenue distribution.** When the model earns revenue at serving time, who is paid? In a centralized lab the answer is: only the company that owns the weights.

Each failure is a missing audit trail. Each is solvable by binding the trail to a consensus ledger. The architecture proposed here—one chain per model—is the smallest design that makes all four trails first-class citizens.

1.1 Why one chain per LLM, not one chain for all LLMs

A single chain holding state for every model creates state-explosion (one weight checkpoint of an 80B model is ≈ 160 GB) and mixes economic incentives across unrelated communities. The contributors to **zen4-coder** have no shared interest with the contributors to **zen4-omni-vision**. Treating each model as a sovereign appchain—following the *Quasar-Native App Stack* pattern (LP-133)—isolates state, isolates economics, and isolates governance, while still letting models share security through QUASAR 3.0 settlement (LP-020).

A per-LLM chain is the minimum unit at which:

- training data, compute receipts, and research artifacts share a coherent dependency graph;
- contributor reward distributions can be specified and enforced without renegotiating with unrelated communities;
- the model’s lifecycle—from initial pretraining shard to deprecation snapshot—fits inside a single state machine.

1.2 Position vs centralized AI labs

OpenAI, Anthropic, and Google operate the closed end of the spectrum: closed weights, closed data, closed contributor graph, closed revenue. Centralized model marketplaces (Hugging Face, Replicate, Together AI) host open weights but do not record provenance and cannot prove which compute ran which step.

The per-LLM chain occupies a strict superset of capabilities:

- **vs closed labs:** open weights, open data lineage, open contributor graph, open revenue rules;
- **vs open marketplaces:** cryptographic provenance, attested compute, deterministic reproduction, on-chain revenue distribution.

The HANZO AI Chain [?] runs the marketplace—listing, discovery, paid inference, commerce. The ZOO per-LLM chains run the upstream lab—training, research, contributor attribution, capacity sourcing. The split is orthogonal: HANZO answers “where do I rent inference?”; ZOO answers “who built this model and how?”.

1.3 Contributions

1. A per-LLM chain specification atop the QUASAR-Native App Stack (LP-133), with state schemas for weight checkpoints, training-data Merkle commitments, attested compute receipts, and contributor attribution graphs (§2).
2. A unified training protocol using BLOCK-STM (LP-010) for deterministic ordering of gradient submissions and QUASARGPU (LP-132) for capacity scheduling (§3).
3. An on-chain research artifact registry: every architecture, hyperparameter, ablation result, and benchmark commitment is reproducible from chain state (§4).
4. A pooled GPU-hosting protocol with stake-weighted slashing for SLA breaches, federated through LUX Cloud (LP-136) (§5).
5. Documentation of the canonical ZEN family deployment—eighteen models, each on its own chain (§6).
6. Cross-chain protocols against LUX C-Chain, the HANZO AI Chain, LUX F-Chain (LP-013), and LUX B-Chain (LP-134) (§7).

2 Architecture

2.1 Settlement on Quasar 3.0

Every per-LLM chain is a QUASAR-certified appchain following the QUASAR-Native App Stack pattern (LP-133). Settlement, finality, and Byzantine guarantees come from QUASAR 3.0 (LP-020); the appchain provides only the model-specific state machine. QUASAR 3.0 finalizes the chain’s headers within sub-second bounds and provides post-quantum security via the QUASAR consensus family.

The chain identifier is a deterministic function of the model’s canonical identifier:

$$\text{chainID} = \text{keccak256}(\text{"zoo://llm/" } \parallel \text{model.canonical_id}) \bmod 2^{32} \quad (1)$$

For **zen4-coder-flash**, the canonical identifier is **zenlm/zen4-coder-flash@v1**. This binding ensures one chain per (model, major-version) pair and forbids two chains claiming the same model.

2.2 State schema

A per-LLM chain’s world state is partitioned into four trees, all rooted into the chain’s state commitment:

Tree	Key	Value
weights	(epoch, shard_id)	shard hash, parameter count, dtype, source job receipt

Tree	Key	Value
data	example_id	content hash, license tag, contributor address, redaction status
receipts	job_id	operator address, GPU model, hours, TEE quote (A-Chain anchor), loss curve hash
attribution	contributor	weighted edge list to (data, weights, research) nodes

2.2.1 Weights tree

Each training epoch produces a weight checkpoint. The checkpoint is content-chunked into shards (default 4 GiB, sufficient for FP16 storage of ≤ 2 B parameters per shard) and a Merkle tree is built over shard hashes. Only the root and shard metadata enter chain state; bulk shard storage is offloaded to content-addressable storage with the chain holding integrity proofs.

2.2.2 Training-data tree

Every training example referenced by any committed gradient step has a record. The record commits to the example’s content hash, license, and contributor identity. Examples may be *redacted* (revoked by contributor) without altering historical receipts, by setting the redaction flag and replacing the value commitment with a redaction proof.

2.2.3 Compute-receipts tree

Every gradient step accepted into a checkpoint is backed by a receipt: the operator address, the GPU hardware, the wall-clock duration, and a TEE attestation quote. TEE quotes are not stored on the per-LLM chain directly; the chain stores a receipt-id and the chain’s bridge to LUX A-Chain (LP-134) verifies the quote, anchors it, and publishes a one-bit attestation result back to the per-LLM chain.

2.2.4 Attribution tree

A weighted directed graph: contributor identity $\rightarrow$ (data nodes, weights nodes, research nodes), with edge weights determined by the chain’s attribution policy (§3). Inference revenue distribution (paid by the HANZO AI Chain when inference is purchased) follows this graph.

2.3 Validator set

A per-LLM chain inherits the QUASAR validator set by default: any QUASAR 3.0 validator may produce blocks for the chain subject to chain-specific stake. Per-LLM chains may also opt into a *constrained validator subset* (e.g., only validators that have committed GPU capacity to the chain’s pool), trading decentralization for higher capacity utilization. The constraint policy is part of the chain’s genesis parameters.

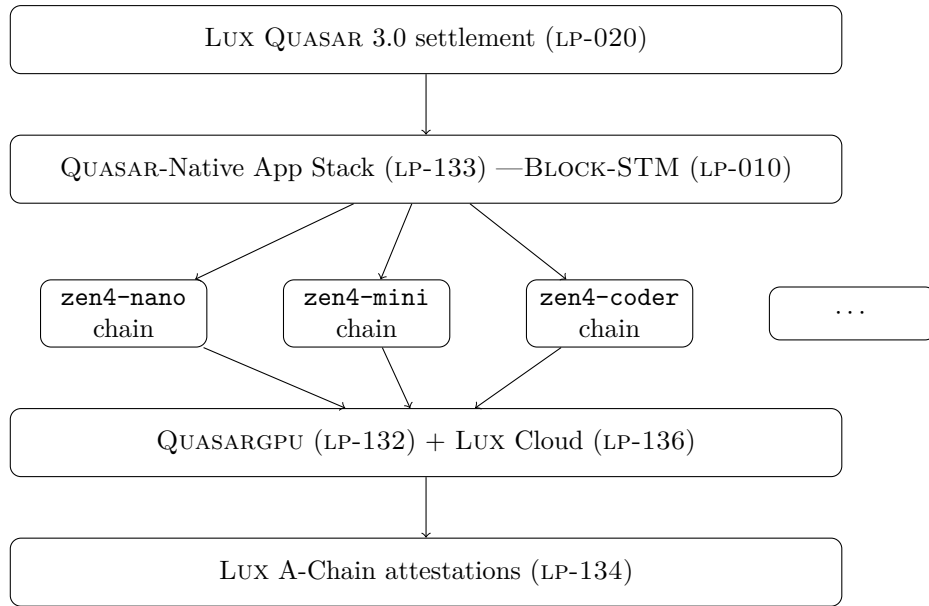
2.4 Native instructions

The chain exposes seven native instructions beyond the standard BLOCK-STM (LP-010) execution surface:

- `ClaimShard(epoch, shard_id)` — operator claims a training shard.
- `SubmitGradient(job_id, grad_hash, loss, tee_quote_id)` — operator submits gradient.
- `MergeCheckpoint(epoch, root)` — aggregator publishes new checkpoint root.
- `RegisterArtifact(kind, hash, metadata)` — publish architecture, ablation, or benchmark.
- `CommitData(example_id, content_hash, license, contributor)` — record training data.
- `Redact(example_id, proof)` — contributor revokes an example.
- `DistributeRevenue(epoch, total)` — pay revenue across attribution graph.

These instructions are precompiled in the chain’s execution layer; user-defined contracts (ERC-20 reward tokens, governance, etc.) coexist as ordinary EVM contracts.

2.5 System diagram



3 Unified Training

3.1 Training as transactions

Training a model on a per-LLM chain is not an off-chain process whose results are imported. It is the chain’s primary economic activity. Every gradient step is a transaction; every checkpoint is the result of executing those transactions in consensus order. BLOCK-STM (LP-010) provides parallel optimistic execution with deterministic ordering, which is exactly the property training requires: anyone replaying the chain reproduces bit-identical checkpoints.

3.2 Training-step transaction

A single training step is the four-instruction sequence:

The expected loss band is parameterized at chain genesis. A loss outside the band rejects the gradient and slashes a configurable fraction of the operator’s stake; this is the chain’s defense against operators submitting arbitrary or adversarial gradients.

Algorithm 1 Per-Step Training Cycle

- 1: `ClaimShard(epoch, shard_id)` — operator stakes claim
 - 2: Operator runs forward+backward pass on shard inside TEE
 - 3: `SubmitGradient(job_id, grad_hash, loss, tee_quote_id)`
 - 4: Chain validates: stake sufficient, TEE quote anchored, loss within expected band
 - 5: `MergeCheckpoint(epoch, root)` once $\geq t$ gradients accepted for epoch
-

3.3 Determinism and reproduction

For training to be reproducible from chain state alone, four things must be deterministic: shard partitioning, gradient ordering, aggregation, and floating-point reduction. The first three are guaranteed by BLOCK-STM sequencing of the four-instruction protocol. The fourth—floating-point—is enforced by requiring all training jobs to use a chain-specified deterministic numerics profile (e.g., NCCL with `CUBLAS_WORKSPACE_CONFIG=:4096:8`, fp32 master weights).

Proposition 3.1 (Reproducibility). *Given the genesis state and the canonical transaction sequence of a per-LLM chain, any party with sufficient compute can reconstruct every weight checkpoint bit-for-bit, modulo the deterministic numerics profile recorded at genesis.*

This is not a theoretical property. It is the basis on which validators verify that a checkpoint matches the gradient transactions that produced it. Without it, the chain cannot detect operators who submit fabricated checkpoints.

3.4 Capacity scheduling via Quasargpu

QUASARGPU (LP-132) is the deterministic GPU scheduling adapter. The per-LLM chain emits capacity requests; QUASARGPU assigns shards to operators in the chain’s pool based on stake-weighted lottery, geo-locality, and announced GPU class. The assignment itself is a deterministic function of chain state, so an attacker cannot manipulate which operator gets which shard.

3.5 Aggregation

Aggregation is the merge of t accepted gradients into the next checkpoint. Two aggregation modes are supported:

- **Synchronous SGD-equivalent:** when t matches the global batch size, aggregation is plain mean. Loss curves match a centralized run identically.
- **Asynchronous with bounded staleness:** gradients accepted within a staleness window are aggregated; stale gradients beyond the window are slashed (not just rejected). This trades reproducibility for throughput on large chains.

The mode is chain-genesis configuration. ZEN models default to synchronous SGD-equivalent for the consumer line and bounded staleness for the frontier line.

3.6 Attribution from gradient receipts

The attribution tree is updated atomically with every accepted gradient. The default policy is *shard-weighted contribution*:

$$w_{\text{op} \rightarrow \text{epoch}} += \frac{\|\nabla_{\text{op}}\|_2}{\sum_{\text{op}'} \|\nabla_{\text{op}'}\|_2} \quad (2)$$

Other policies (compute-time-weighted, loss-improvement-weighted) may be specified at genesis or amended via on-chain governance. Inference revenue distribution downstream uses these weights as the payout vector for the corresponding epoch.

3.7 Slashing

Three slashing conditions:

1. **Out-of-band loss:** gradient produces a loss outside the expected band. Slash 5% of stake.
2. **Invalid tee quote:** A-Chain (LP-134) bridge returns a failed attestation. Slash 25%.
3. **Equivocation:** operator submits two different gradients for the same shard claim. Slash 100%.

4 Unified Research

4.1 Research as on-chain artifacts

Research outputs that influence the model—architecture variants, hyperparameter choices, ablation results, benchmark scores—are stored as first-class on-chain artifacts. The `RegisterArtifact` instruction commits an artifact of one of four kinds:

Kind	Hashed object	Use
architecture	model definition file	defines the structure trained in subsequent epochs
hyperparams	training config (yaml/json)	locks the schedule used for the run
ablation	experiment log + result tuple	records what was tried and the outcome
benchmark	dataset commitment + scores	committed evaluation result

4.2 Research artifact registry contract

The chain ships with a `ResearchRegistry` precompile-backed contract whose state-changing methods are restricted to addresses on the model’s contributor allowlist (a list managed by the model’s governance). Reads are unrestricted.

```

1 interface IResearchRegistry {
2     function commitArchitecture(bytes32 hash, bytes calldata meta)
      external;
3     function commitHyperparams(uint256 epoch, bytes32 hash) external;
4     function commitAblation(bytes32 parentArch, bytes32 hash, int256
      deltaScore) external;
5     function commitBenchmark(bytes32 dataset, bytes32 scoresHash) external
      ;
6     function get(bytes32 artifactId) external view returns (Artifact
      memory);
7 }

```

Listing 1: Registry interface (excerpt)

4.3 Reproducibility from artifacts

Because the artifact registry, the data tree, and the receipts tree are all on the same chain, reproducing any reported result is mechanical:

1. Look up the architecture artifact and hyperparameters for the target epoch.
2. Walk the receipts tree to enumerate the gradient transactions that built the epoch’s checkpoint.
3. Replay deterministically per §3.
4. Look up the benchmark artifact and re-run on the committed dataset.

The chain itself does not run benchmarks (they require model inference, which the HANZO AI Chain serves), but it provides a tamper-evident bibliography for any reported number.

4.4 Citation graph

Ablations form a directed graph rooted at architecture artifacts. The graph is queryable via standard subgraph indexers; the chain emits typed events on every commit. This produces a transparent research history: any claim of the form “architecture X improves Y by δ ” has a chain transaction that records the parent artifact, the child artifact, and the delta.

4.5 Research credit attribution

Research artifacts contribute to the attribution tree the same way training shards do, but with a configurable multiplier. The default policy:

$$w_{\text{author} \rightarrow \text{epoch}} += \begin{cases} \alpha_{\text{arch}} \cdot \mathbf{1}[\text{arch adopted}] & \text{if architecture} \\ \alpha_{\text{hp}} \cdot \mathbf{1}[\text{hp adopted}] & \text{if hyperparams} \\ \alpha_{\text{abl}} \cdot \max(0, \delta) & \text{if ablation} \\ \alpha_{\text{bench}} \cdot \log(1 + \text{citations}) & \text{if benchmark} \end{cases} \quad (3)$$

with $\alpha_{\text{arch}} = 1.0$, $\alpha_{\text{hp}} = 0.1$, $\alpha_{\text{abl}} = 0.5$, $\alpha_{\text{bench}} = 0.2$ as defaults. The values are amendable by chain governance.

5 Pooled GPU Hosting

5.1 Capacity is staked, not just announced

Operators commit GPU capacity to a per-LLM chain’s pool by locking stake proportional to announced capacity. Announcement-only schemes (such as legacy compute marketplaces) cannot punish no-shows. Stake-backed announcements can: when an operator fails to fulfill a claimed shard within the SLA window, stake is slashed.

Definition 5.1 (Capacity commitment). *A capacity commitment is the tuple $(\text{operator}, \text{gpu_class}, \text{count}, \text{stake})$ such that*

$$\text{stake} \geq \kappa \cdot \text{count} \cdot \text{price}(\text{gpu_class})$$

where κ is the chain’s capacity-stake multiplier (default $\kappa = 2$, meaning operators stake 2x the spot price of one hour of capacity).

5.2 Federation through Lux Cloud

LUX Cloud (LP-136) is the broader cloud-capacity layer that aggregates GPU, CPU, and storage offers from operators across the LUX ecosystem. Per-LLM chains specialize in AI training workloads and federate through LUX Cloud rather than running their own discovery layer:

- **Discovery:** LUX Cloud exposes the operator catalog, hardware claims, and price feeds.
- **Specialization:** per-LLM chains accept only operators whose announced GPU class meets the model’s minimum (e.g., ≥ 80 GB HBM for ≥ 30 B-parameter models).
- **Settlement:** capacity payments settle on the per-LLM chain in its native reward token, not on LUX Cloud directly. LUX Cloud is a discovery and federation layer, not a settlement layer for AI workloads.

5.3 SLA and slashing

Per shard claim, the SLA is parameterized by:

- **deadline:** wall-clock time to submit gradient.
- **loss_band:** $[\ell_{\min}, \ell_{\max}]$ acceptable loss range.
- **tee_required:** whether TEE attestation is mandatory (default true for ≥ 8 B models).

Breach conditions and slashing fractions are specified in §3. Slashed stake is partially burned and partially redistributed to validators that signed the slashing block.

5.4 Pricing

Capacity is paid out per accepted gradient, not per claimed shard. The price formula:

$$\text{payout} = \text{base_rate}(\text{gpu_class}) \cdot \text{flops_consumed} \cdot \mu(\text{loss_improvement}) \quad (4)$$

where μ is a multiplier rewarding operators whose gradients drive larger loss decreases. The base rate is set by the chain’s price oracle (which may peg to LUX Cloud spot prices).

5.5 Hosting beyond training

Pooled GPU hosting is not exclusive to training. The same operator pool serves:

- **Inference for the Hanzo AI Chain.** When a user buys inference through HANZO, the inference workload routes to operators registered on the model’s chain. The operator selection is stake-weighted; payouts settle through cross-chain messaging (§7).
- **Evaluation runs.** Benchmark commits in the research registry can include execution receipts pointing to operators that ran the evaluation.
- **Private inference via F-Chain.** Encrypted inference (LP-013) routes through F-Chain but uses the same operator pool for the underlying compute, with FHE wrappers applied at the edge.

5.6 Operator economics

An operator running a single H100 80 GB at 90% utilization across one year of staking is the reference economic unit. Sustained payouts are determined by the chain’s training schedule, the inference demand routed from HANZO, and the operator’s stake share. Detailed economics are model-specific and recorded in each chain’s genesis configuration.

6 The Zen Model Family

6.1 Canonical deployment

The ZEN family¹ is the canonical ZOO per-LLM chain deployment. Each released ZEN model has a sovereign chain identified by the canonical id `zenlm/<model>@<major>`.

The lineup below is the current `zen4` generation deployed at chain genesis (2025-12-25) plus the in-development `zen5` line whose chains are reserved.

6.2 Small line (Qwen3.5 base, Gated DeltaNet, 262K context)

Model	Total	Active	License	Notes
zen4-nano	0.8B	0.8B	Apache 2.0	abliterated; edge
zen4-micro	2B	2B	Apache 2.0	abliterated; mobile
zen4-mini	4B	4B	Apache 2.0	general-purpose
zen4	9B	9B	Apache 2.0	standard

6.3 Medium line (Qwen3.5 base, mixed dense/MoE)

Model	Total	Active	License	Notes
zen4-pro	27B	27B	Apache 2.0	professional
zen4-max	35B MoE	3B	Apache 2.0	MoE flagship (consumer)
zen4-pro-max	80B MoE	3B	Apache 2.0	ultimate consumer
zen4-mega	122B MoE	10B	Apache 2.0	higher-active MoE

6.4 Coder line

Model	Total	Active	License	Notes
zen4-coder-flash	31B MoE	3B	MIT	GLM-4.7-Flash base
zen4-coder	80B MoE	3B	Apache 2.0	512 experts
zen4-coder-pro	355B	355B	MIT	cloud-only

6.5 Frontier line

Model	Total	Active	License	Notes
zen4-thunder	230B MoE	10B	MIT	MiniMax-M2.5 base
zen4-storm	456B MoE	45B	MIT	MiniMax-M1 base

¹Source repositories: `~/work/zen` (training), `huggingface.co/zenlm` (releases).

Model	Total	Active	License	Notes
zen4-titan	744B MoE	40B	open	GLM-5 base
zen4-ultra	1.04T MoE	32B	open	Kimi K2.5 base
zen4-ultra-gguf	1.04T MoE	32B	open	K2.5 abliterated GGUF
zen4-ultra-max	1T+ MoE	50B	MIT	Ling-1T base

6.6 Vision-language sub-family

The **zen-vl** line (released earlier and rolled into the per-LLM chain framework at activation) provides image+text models on Qwen3-VL bases at 4B, 8B, and 30B/MoE scales, each with an **-instruct** and **-agent** variant. Each VL model receives its own chain.

6.7 Speech sub-family

zen-tts (text-to-speech, multilingual, 24kHz output, voices: Cherry, Serena, Ethan, Chelsie, Momo) and **zen-omni** (unified text+vision+audio) each have their own chain. Modality-specific instructions extend the standard schema with audio-clip provenance records for **zen-tts** and per-modality data trees for **zen-omni**.

6.8 Reranker, embedding, and specialty chains

- **zen-reranker** (ZIP-002) — reranking model, dedicated chain.
- **zen-embedding** — embedding model.
- **zen-guard**, **zen-guard-stream**, **zen-guard-gen** — safety classifiers; share a single guard chain due to interlocking training.
- **zen-scribe**, **zen-blog**, **zen-translator**, **zen-dub**, **zen-foley**, **zen-musician**, **zen-3d**, **zen-artist**, **zen-designer**, **zen-director**, **zen-video**, **zen-world**, **zen-voyager** — specialty chains, each with its own training history and contributor graph.

6.9 Lineage

The base-model lineage table below records the upstream foundation each ZEN model derives from. The lineage is itself an artifact registered on the per-LLM chain (kind: **architecture**, with parent pointers to upstream sources where licensing permits). Brand policy: public docs do not name upstream organizations; the chain records them as cryptographic commitments only.

Zen model	Upstream foundation (chain-recorded)
zen4-nano – zen4	Qwen3.5 (Gated DeltaNet)
zen4-pro – zen4-mega	Qwen3.5 abliterated lineage
zen4-coder-flash	GLM-4.7-Flash
zen4-coder	Qwen3-Coder-Next
zen4-coder-pro	GLM-4.7
zen4-thunder, zen4-storm	MiniMax M-series
zen4-titan	GLM-5
zen4-ultra, zen4-ultra-gguf	Kimi K2.5

Zen model	Upstream foundation (chain-recorded)
zen4-ultra-max	Ling-1T
zen-vl-*	Qwen3-VL
zen-omni	Qwen3-Omni

6.10 License and attribution

Each ZEN chain inherits the upstream license (Apache 2.0 or MIT) and adds the chain-recorded contributor attribution graph as a binding revenue policy. Inference revenue routed from the HANZO AI Chain to the ZEN chain is distributed per-epoch according to the attribution tree at the time the inference was billed.

7 Cross-Chain Interaction

A per-LLM chain is one node in a graph of chains. Every meaningful workflow—training, inference, attestation, payment—crosses at least one chain boundary. The interaction patterns below are the standard set.

7.1 Lux C-Chain (general EVM)

C-Chain hosts the general EVM contract surface used by external dApps. Two flows:

- **Outbound (per-LLM $\rightarrow$ C-Chain).** The chain emits an event when a checkpoint is finalized; a C-Chain registry contract subscribes via Warp Messaging and updates a public model index.
- **Inbound (C-Chain $\rightarrow$ per-LLM).** An external grant program funds training bounties on C-Chain in stablecoins; a bridge contract transfers the bounty to the per-LLM chain’s bounty escrow.

7.2 Hanzo AI Chain (inference marketplace)

The HANZO AI Chain is the marketplace for inference and AI commerce. The cross-chain protocol with per-LLM chains:

1. Listing: a model is listed for inference on HANZO by submitting a chain-cross-reference proof: “the per-LLM chain at `chainID = X` has finalized weights at root `R` as of epoch `e`”.
2. Inference: a user pays HANZO for inference; HANZO routes the workload to operators registered on the ZEN chain; HANZO settles operator payment in its native unit.
3. Revenue distribution: a fraction of the inference revenue (chain-specified, default 30%) is forwarded to the per-LLM chain via cross-chain transfer and distributed across the attribution tree.

The split is orthogonal: HANZO runs the storefront and inference orchestration; ZOO per-LLM chains run the lab and attribution. Neither chain duplicates the other’s state.

7.3 Lux F-Chain (encrypted inference)

F-Chain (LP-013) provides FHE-protected inference for queries that must remain private to the chain operators. The integration:

- User encrypts query under the chain’s FHE public key.

- F-Chain orchestrates ciphertext-domain inference using operator pool registered on the model’s per-LLM chain.
- Decryption key remains with the user; inference output is returned in ciphertext.
- Settlement: F-Chain pays operators; per-LLM attribution credit accrues to the originating model’s contributors.

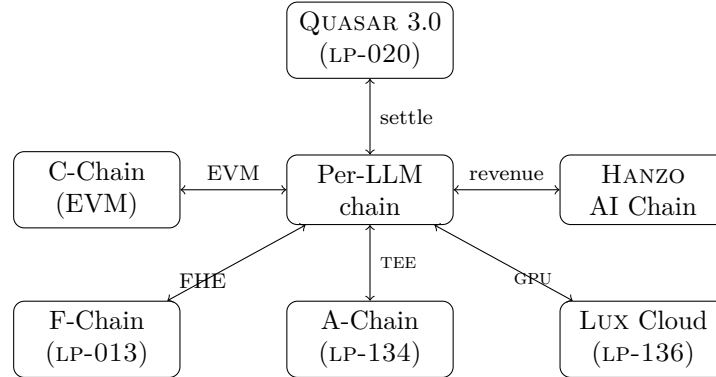
7.4 Lux A-Chain and B-Chain (lp-134)

A-Chain anchors TEE attestations. Every gradient submission’s quote-id is verified by an A-Chain bridge before the per-LLM chain admits the gradient. B-Chain provides the bridge layer between per-LLM chains and external chains (Ethereum, Solana, etc.) for ERC-20 reward token wraps and cross-chain bounty receipts.

7.5 Lux Cloud (lp-136)

LUX Cloud is the federation layer for cloud capacity. The integration is documented in §5. Operationally, every per-LLM chain registers itself with LUX Cloud’s discovery service so that operators can find and stake on chains they wish to support.

7.6 Topology summary



8 Token Economics

8.1 Two-token model

Each per-LLM chain operates with two distinct economic instruments:

1. **ZOO** (network-wide) — the ZOO Network governance and staking token, fixed supply, used for cross-chain validator stake and DAO governance. ZOO is shared across all per-LLM chains and is not minted by any individual chain.
2. **\$<MODEL>** (chain-specific) — a per-chain reward unit (e.g., \$ZEN4-CODER, \$ZEN4-MAX). Minted by the chain to pay training, hosting, and research contributors.

The split mirrors the established ZOO pattern: \$ZOO is a fixed-supply ecosystem token; per-application reward tokens are emitted at the application layer to compensate work specific to that application. Per-LLM chains apply this pattern to model lifecycle work.

8.2 Per-chain token supply

A per-LLM chain’s reward token has a fixed cap of 10^9 units. The default allocation:

Bucket	Share	Purpose
Training rewards	50%	Distributed to gradient submitters per attribution
Hosting rewards	20%	Distributed to inference operators
Research rewards	10%	Distributed to artifact authors
Data contributors	10%	Distributed to contributors of training data
DAO treasury	5%	On-chain governance treasury
DEX liquidity	5%	Initial liquidity on ZOO DEX

8.3 Emission schedule

Emissions follow a halving schedule. Default parameters:

- Block reward r_0 : chain genesis.
- Halving period: 2^{22} blocks (≈ 13 months at 1 s blocks).
- Total emissions converge to the fixed cap.

The emission rate per block is split across buckets in proportion to the allocation. For example, in any given block, 50% of the emitted reward token enters the training-rewards distribution pool and is paid out per accepted gradient that block.

8.4 Inference revenue split

When the HANZO AI Chain forwards a fraction (default 30%) of inference revenue to the per-LLM chain, the funds enter the chain in stable units (USDC-equivalent on LUX) and are distributed without conversion to the chain’s reward token:

$$\text{contributor share} = \text{revenue} \cdot \frac{w_{\text{contributor}}}{\sum_c w_c} \quad (5)$$

Distribution is per-epoch: the attribution graph snapshot at billing time governs the payout vector for that epoch’s revenue.

8.5 Staking and governance

Validators that produce blocks for a per-LLM chain stake \$ZOO; operators that commit GPU capacity stake \$ZOO plus a per-chain capacity stake; contributors hold the chain’s reward token to vote on chain governance (slashing parameter changes, attribution policy amendments, license amendments).

The chain’s reward token is non-transferable for an initial vesting period (default 90 days) to prevent spot-flipping of training rewards.

8.6 Slashing economics

Slashed stake is split: 50% burned, 30% redistributed to the validator that signed the slashing block, 20% returned to the chain’s DAO treasury. The split discourages collusive slashing while still giving validators an incentive to detect violations.

9 Privacy and Provenance

9.1 Selective disclosure of training data

The training-data tree commits to (content hash, license tag, contributor). Disclosure of the example content itself is not required for the commitment to be valid. Three disclosure modes are supported:

1. **Public:** content is published to content-addressable storage; anyone with the hash can fetch.
2. **Permissioned:** content is published encrypted under the contributor’s policy key; consumers prove eligibility off-chain.
3. **Private:** content is held by the contributor; the chain holds only the commitment and a zero-knowledge proof of license.

The chain stores a *disclosure flag* per record. Inference downstream may filter to public-only data when the use case requires (e.g., open-source model card generation).

9.2 Differential privacy on gradient submission

Operators may submit gradients with calibrated noise added to satisfy (ϵ, δ) -differential privacy. The chain records the noise parameters; the loss-band check is widened proportionally. Models with strong privacy requirements (medical, legal) genesis-configure all gradient submissions to require DP noise above a threshold.

The mechanism is straightforward Gaussian noise addition:

$$\tilde{g} = g + \mathcal{N}(0, \sigma^2 I), \quad \sigma = \frac{C \sqrt{2 \ln(1.25/\delta)}}{\epsilon} \quad (6)$$

where C is the gradient clip norm. The chain enforces C through the bounded-norm gradient submission instruction; an operator submitting an over-norm gradient is slashed.

9.3 TEE attestation as compute provenance

TEE attestations (anchored on LUX A-Chain per LP-134) provide cryptographic evidence that the gradient was computed inside an enclave running the chain-specified training image. The evidence chain:

1. Genesis pins the SHA-256 hash of the training image.
2. Operator’s GPU runs the image inside a TEE (NVIDIA H100 Confidential Computing or AMD MI300X SEV-SNP).
3. TEE produces a quote signed by the hardware root of trust.
4. A-Chain verifies the quote and publishes a binary attestation result.
5. Per-LLM chain admits the gradient only on positive attestation.

9.4 Redaction and right-to-revoke

A contributor may revoke a previously committed training example. The redaction transaction:

1. Sets the redaction flag on the `data` tree entry.
2. Replaces the content commitment with a zero-knowledge proof that the original committer holds the same key.
3. Triggers downstream subscribers (model serving operators) to remove the corresponding example from any active replicas.

Crucially, redaction does not erase historical receipts: the gradient transactions that consumed the now-redacted example remain on chain. Reproducing the model from chain state would require the redacted data, which the redactor refuses to release. The chain’s reproducibility guarantee is therefore conditional on data availability; redaction trades reproducibility for contributor sovereignty. This is the documented tradeoff, not a bug.

9.5 Reading the provenance graph

External consumers (auditors, regulators, downstream model users) read provenance via subgraph indexers. Standard queries:

- “What licenses appear in the training data of **zen4-coder** as of epoch e ?”
- “Which contributors hold $> 1\%$ attribution in **zen4-pro-max**?”
- “Which gradient submissions failed TEE attestation in the last 30 days?”
- “Was example h used after redaction event r ?”

Each is answerable from the per-LLM chain’s state with no off-chain trust assumption beyond the QUASAR consensus that finalized the chain’s headers.

10 Activation

10.1 Spec freeze and activation timeline

Date	Milestone
2025-12-15	QUASAR 3.0 spec freeze (LP-020). Per-LLM chain spec frozen alongside.
2025-12-16 – 2025-12-22	Genesis dry-runs on testnet for the zen4 family. Validators rehearse BLOCK-STM ordering, A-Chain bridge handshake, and LUX Cloud federation.
2025-12-23 – 2025-12-24	Final genesis state generated. Validator set seeded. Operator pool stake-locked.
2025-12-25	QUASAR 3.0 activation. zen4 per-LLM chains launch with checkpoints derived from upstream HF weights. First on-chain training epoch begins.

10.2 Genesis state

For each ZEN model in §6, genesis state contains:

- The HF source commit pinned as the initial weights root.
- The architecture artifact for the upstream foundation, registered as research artifact zero.
- An empty receipts tree (no training has occurred on chain yet).
- An empty attribution tree (initial weights are imported, not earned on chain).
- The reward token contract deployed and the DAO treasury allocated.

A genesis import is itself a transaction signed by the ZOO multisig that holds upstream license attestations. Subsequent training accrues attribution to on-chain contributors as defined in §3.

10.3 Operator onboarding

Operators wishing to commit capacity at activation must:

1. Register an account on LUX Cloud (LP-136).
2. Submit hardware claim (GPU class, count, geo, network).
3. Stake-lock per the chain's κ multiplier.
4. Pass an attestation handshake against A-Chain (LP-134).

The onboarding window closes 24 h before activation; post-activation onboarding is continuous but new operators wait one epoch before receiving shard claims (this is a slashing-safety buffer, not a centralization choice).

10.4 Post-activation roadmap

- **Q1 2026 (post-activation):** **zen4** identity LoRA fine-tunes are first-on-chain training runs; results validate the protocol against existing centralized fine-tunes.
- **Q2 2026:** **zen5** chains genesis. **zen5** introduces MoDE (Mixture of Diverse Experts) and is the first ZEN generation trained natively on chain rather than imported from upstream.
- **Q3 2026:** cross-chain inference revenue routing from HANZO AI Chain to ZOO per-LLM chains live in production.
- **Q4 2026:** F-Chain (LP-013) FHE inference live for **zen-omni** and **zen-v1-*** chains.

10.5 Conclusion

Per-LLM chains are the smallest design that binds training data, compute, research, and revenue into a single audit-eligible ledger. They are not a generic blockchain hosting model files; they are an execution environment in which the lifecycle of a single model is the chain's reason to exist. The ZEN family demonstrates the design at scale: eighteen models, eighteen sovereign chains, one settlement layer. QUASAR 3.0 (LP-020) provides the security; the QUASAR-Native App Stack (LP-133) provides the substrate; QUASARGPU (LP-132) provides scheduling; A-Chain (LP-134) provides attestation; LUX Cloud (LP-136) provides capacity federation. The ZOO per-LLM chains compose these into the open foundation-model lab that centralized labs cannot match.

Activation: 2025-12-25.